Closed-form chance constraints for fast multi-UAV replanning under uncertain execution

Cao-Phuc Ha[a], Anh Nguyen Vu Minh[a], Phu-Nguyen Le[b,*]

[a]*Department of Artificial Intelligence, FPT University, Ho Chi Minh City, Vietnam*
[b]*Faculty of Electrical and Electronics Engineering, Vietnam Aerospace University, 104 Nguyen Van Troi Street, Phu Nhuan Ward, Ho Chi Minh City, 700000, Vietnam*

Abstract

Multi-UAV replanners usually plan for nominal execution. Under wind, speed and service-time errors, such plans promise finish times that the vehicles do not achieve, and tasks near the end of a route are served late. This paper proposes AnytimeSwarm-CC, a replanner that plans against a chance constraint on the finish time of every task. Because the mean and the spread of a calibrated finish-time error grow linearly with the lead time, the chance constraint reduces exactly to an effective deadline per task. The margin therefore grows along a route, and the fast repair and adaptive large neighbourhood search of the nominal model are reused unchanged. A nominal fallback keeps the tasks that the constraint would give up. The replanner was evaluated in pre-registered studies on held-out missions of a kinematic simulator with disturbances and communication latency. In 48 600 runs, the chance-constrained model served 1.35 to 2.87 percentage points more reward than each of six baselines, among them metareasoning stopping rules and a constant time buffer (Holm-adjusted $p < 0.001$). Ablations attributed the gain to the planning model rather than to the stopping rule. On new seeds, the fallback added 1.97 percentage points and served 12.8% more reward than a conventional event-triggered planner, at the price of more late tasks. In PX4 software-in-the-loop flights with Gazebo and ROS 2, the replanner served 25% fewer tasks late, but the utility differences on 30 missions stayed within the noise.

Keywords: multi-UAV task allocation, dynamic vehicle routing, chance-constrained planning, execution uncertainty, anytime planning, PX4 software-in-the-loop

1. Introduction

Teams of unmanned aerial vehicles (UAVs) are used for inspection, delivery and monitoring. During a mission, new tasks may appear, a vehicle may fail, or a deadline may be moved forward. A central planner then repairs the routes of the whole fleet within a fraction of a second, while some UAVs may already wait for new instructions. Dynamic vehicle routing and multi-robot task allocation offer many methods for this repair [1–3], and metaheuristics such as adaptive large neighbourhood search (ALNS) [4] find good plans within such budgets.

Most of these replanners plan for nominal execution. When UAVs meet wind, fly slower than commanded or take longer at a task, part of the slack that the plan promises is consumed by errors that the planner does not see, and tasks near the end of a route are served late. The error grows with the time until a task is reached, so a constant time margin is too large for early tasks and too small for late ones. Stochastic and robust

*Corresponding author.
Email addresses: caophucai@gmail.com (Cao-Phuc Ha), anhnvmse203412@fpt.edu.vn (Anh Nguyen Vu Minh), nguyenlp@vaa.edu.vn (Phu-Nguyen Le)

routing protect schedules against such errors [5, 6], and chance-constrained allocation bounds the probability of failure [7]. These models are usually solved by sampling or by exact methods that are too slow for the time available between two events. A second line of work treats the computation time of an anytime planner as a decision [8–11]. It is not known which of the two, the planning model or the control of computation, matters more for a replanner that faces uncertain execution.

This paper proposes AnytimeSwarm-CC, a replanner that plans against a chance constraint on the finish time of every task and keeps the fast heuristics of the nominal model. Its contributions are threefold. The first is a closed-form chance-constrained planning model. A calibrated finish-time error, whose mean and spread grow linearly with the lead time, turns a chance constraint on every task into an effective deadline. The margin thus grows along a route, and the model is solved by the same repair and ALNS as the nominal one, with a negligible overhead per call. A nominal fallback keeps the tasks that the constraint would give up and leaves the guarantee of all other tasks intact. The second is a pre-registered comparison on held-out missions with re-implemented baselines. These include stopping rules from the metareasoning literature [10–12], size-scaled and fixed budgets, a constant time buffer [6] and a conventional event-triggered planner, and ablations separate the effects of the planning model, the stopping rule and the replanning trigger. The third is a software-in-the-loop check with PX4 autopilots [13] in Gazebo [14] and ROS 2 [15], after a rescaling that keeps every dimensionless ratio of the decision problem, together with an estimate of the number of flights that such a check requires.

The results are reported as they came out. In the kinematic tier, the chance-constrained model served 1.35 to 2.87 percentage points more reward than each of six baselines, and ablations traced the gain to the planning model rather than to the stopping rule. The model was too cautious under high disturbances with halved time windows, and the nominal fallback, tested afterwards on new seeds, removed this weakness. In software-in-the-loop flights, the replanner served fewer tasks late, but the utility differences stayed within the noise. An earlier pre-registered study, in which only the stopping rule was made execution-aware, found no significant gain in utility (Section 6.1).

2. Related work

Dynamic task allocation and routing. Multi-robot task allocation [1] and dynamic vehicle routing [2, 3] reassign tasks when information changes. The planning problem considered here is a prize-collecting routing problem with time windows [16, 17]. Large neighbourhood search has been adapted to dynamic routing with time windows [18]. For multi-UAV systems, auction methods such as CBBA reach conflict-free assignments quickly [19]. Recent work handles stochastic task completion with hierarchical policies [20], uncertain execution times with dynamic time slices [21], intermittent communication with uncertainty-gated distributed allocation [22], and events with assignment-preserving or local repair [23, 24]. Event-triggered and periodic allocation have been compared for multi-UAV systems [25], in line with the rescheduling literature [26, 27]. Calvo and Capitan validate a repair-based architecture with PX4 and Gazebo [28], and Ali et al. use ArduPilot software-in-the-loop simulation [29]. Table 1 compares these approaches.

Uncertainty in routing and allocation. Stochastic vehicle routing models travel and service times as random variables [30]. With hard time windows and stochastic service times, a priori routes with recourse [5] and robust formulations over budgeted uncertainty sets [6] protect the schedule against delays, at the price of a harder model. Scenario-based approaches plan against sampled futures [31], and chance-constrained task allocation bounds the probability that a team misses its objectives [7]. The proposed model follows this line. It exploits one structural property, an error whose mean and spread grow linearly with the lead time, under which a per-task chance constraint becomes a deterministic deadline and the heuristics of the nominal model apply unchanged. A constant time buffer, the simplest robust alternative, serves as a baseline.

Table 1: Features of recent approaches to dynamic task allocation and replanning compared with the proposed method. TW: time windows or deadlines; Exec. unc.: execution or duration uncertainty in the planning model; Trigger: replanning trigger policies compared; Stop: when to stop the replanner (computation as a decision); SITL: validation with an autopilot in software-in-the-loop or physics simulation. ✓: yes; ∘: partly; –: no or not reported.

Work	Problem	TW	Exec. unc.	Trigger	Stop	SITL	Architecture
Choudhury et al. [20]	MRTA, time windows	✓	✓	–	–	–	hierarchical
Huang et al. [21]	multi-UAV allocation	–	✓	∘	–	–	–
Calvo and Capitan [28]	long-endurance UAV MRTA	∘	–	–	–	✓	centralised
Hwang et al. [23]	multi-UAV replanning	✓	–	–	–	–	–
Shen et al. [24]	UAV swarm allocation	∘	–	–	–	–	edge
Qian and Fu [22]	multi-UAV allocation	✓	✓	–	–	–	distributed
Cong et al. [25]	multi-UAV allocation	✓	∘	✓	–	∘	–
Ali et al. [29]	multi-UAV allocation	–	–	–	–	✓	decentralised
Svegliato et al. [11]	anytime metareasoning	–	–	–	✓	–	–
Proposed	multi-UAV replanning	✓	✓	✓	✓	✓	centralised

Metareasoning and anytime algorithms. Anytime algorithms [8, 10] and the rational allocation of computation [9, 12] decide how long a planner should compute. Hansen and Zilberstein derive a monitoring policy over a performance profile [32]. Svegliato et al. predict the performance of an anytime algorithm online [11], and Sung et al. learn when an anytime motion planner should stop [33]. Situated temporal planning accounts for the passage of time during planning [34, 35], and deadline-aware search adapts to a fixed deadline [36]. Two of these rules, static performance profiles and online performance prediction, are re-implemented as baselines, so that the value of controlling computation can be compared with the value of the planning model.

3. Problem formulation

A central planner is called whenever a replanning trigger fires. Each UAV keeps the task that it is currently flying to or serving, which determines when and where it becomes available again and how much energy it has left. Each open task j has a location, a release time r_j, a deadline d_j by which its service must be completed, a service duration and a reward w_j. A plan assigns an ordered sequence of open tasks to each UAV, and tasks may remain unassigned, so the planning problem is a prize-collecting routing problem with time windows [16, 17]. Arrival, start and finish times along a route are computed as usual for time windows [37]. A route is feasible if every service ends before its deadline and the UAV returns to the depot with an energy reserve of 10% of its capacity. A plan π is evaluated by

$$J(\pi) = \alpha R(\pi) - \gamma E(\pi) - \delta C(\pi), \tag{1}$$

where R is the reward of the tasks served on time, E the flown distance in kilometres and C the number of tasks that move to another UAV than in the previous plan, with $\alpha = 1$, $\gamma = 0.05$ per kilometre and $\delta = 0.3$. W denotes the total reward of the open tasks.

Computation is not free. A plan computed for τ milliseconds is released $\kappa\tau$ milliseconds later in mission time, where the latency factor κ represents planning computers slower than the evaluation computer. An idle UAV waits for the new plan, so its route starts later, and tasks whose deadline can no longer be met are lost. The plan then reaches the vehicles after a dissemination latency L that the planner does not control.

The planner uses a nominal model with straight legs at the cruise speed of each UAV and nominal service durations. The executed finish time of task j is $F_j = f_j + e_j$, where f_j is the nominal finish time at publication and e_j an execution error caused by acceleration, wind, speed tracking and service-time variability. A task yields its reward only if $F_j \le d_j$. As in dispatching practice, a UAV skips a task at departure if the nominal estimate from its actual state already misses the deadline, and the task returns to the pool. The utility of a

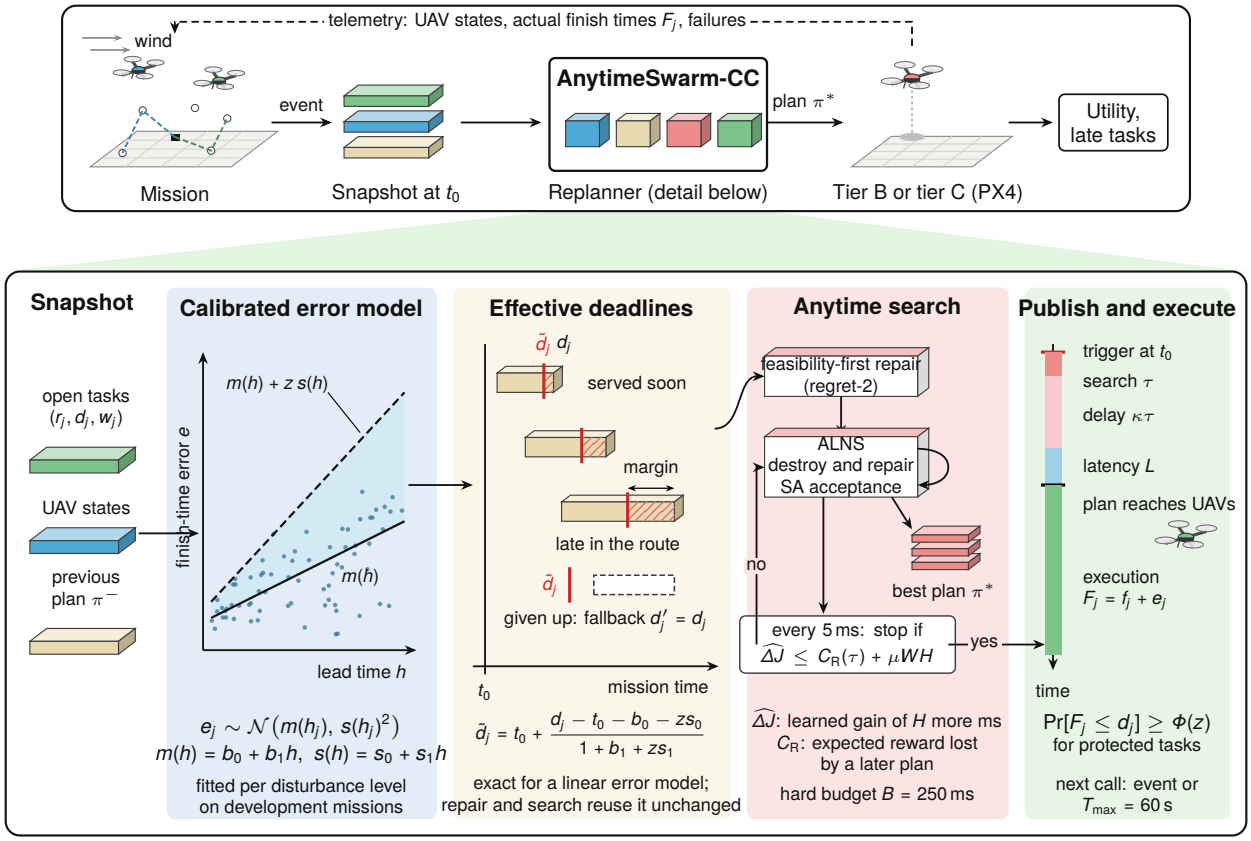


Figure 1: Overview of AnytimeSwarm-CC. Top: the replanning loop between the mission, the planning snapshot, the replanner and the execution tier. Bottom: one replanning call. The calibrated error model gives a margin that grows with the lead time, the chance constraint turns it into an effective deadline per task, the nominal fallback keeps the tasks that no vehicle could otherwise reach, and the anytime search stops when the predicted gain no longer pays for the delay. The plan is released after the emulated planning delay $\kappa\tau$ and the latency L.

mission is the reward of all tasks served on time divided by the total reward of the mission. Three replanning triggers are considered. The event trigger replans on task arrivals, UAV failures and deadline changes, the periodic trigger in addition every T seconds, and the deviation trigger in addition when a finish time deviates from the plan by more than θ seconds. During planning, every UAV finishes its committed task and then waits, and events are queued.

4. Proposed method

AnytimeSwarm-CC takes three decisions in every replanning cycle (Fig. 1, Algorithm 1). The chance-constrained planning model decides which plans are acceptable under the calibrated execution error, the stopping rule decides when to stop computing and publish, and the trigger decides when to replan. A feasibility-first repair and an anytime ALNS produce the plan. Two earlier variants serve as baselines. AnytimeSwarm uses the same stopping rule with a deterministic delay cost on the nominal model, and AnytimeSwarm-R uses an execution-aware delay cost on the nominal model.

4.1. Repair and anytime search

The previous plan is restricted to the open tasks and walked once, keeping a task only if the route up to it is still feasible, and tasks of failed UAVs and new tasks are inserted by parallel regret-2 insertion [4, 38]. This

Algorithm 1 One replanning call of AnytimeSwarm-CC

Require: snapshot at time t_0: committed tasks, open tasks with (r_j, d_j, w_j), UAV states; previous plan π^-; error model (b_0, b_1, s_0, s_1), safety factor z, expected latency $\bar{L}$, latency factor κ

$\tilde{d}_j \leftarrow \min\{d_j,\ t_0 + (d_j - t_0 - b_0 - z s_0)/(1 + b_1 + z s_1)\}$ for every open task ▷ Eq. (4)

$d'_j \leftarrow d_j$ if no UAV can finish j by $\tilde{d}_j$ but one can by d_j, else $d'_j \leftarrow \tilde{d}_j$ ▷ Eq. (5)

$\pi \leftarrow \textsc{FeasibilityFirstRepair}(\pi^-, d')$; $\pi^* \leftarrow \pi$; $\tau \leftarrow 0$

repeat

  destroy and repair π with adaptively chosen operators; accept by simulated annealing

  if $J(\pi) > J(\pi^*)$ **then** $\pi^* \leftarrow \pi$

  every 5 ms: $\widehat{\Delta J} \leftarrow$ learned prediction of the gain in the next H ms

until $\widehat{\Delta J} \le C_{\mathrm{R}}(\tau) + \mu W H$ **or** $\tau \ge B$ ▷ Eq. (6)

publish π^*; it reaches the UAVs at $t_0 + \kappa\tau + L$

next call at the next event or $T_{\max}$ after t_0, whichever comes first

feasibility-first repair yields a publishable plan within milliseconds. ALNS [4] then improves it with four destroy operators (random, worst, related [39] and route-segment removal), four greedy or regret-2 repair operators with and without noise, adaptive operator selection and simulated-annealing acceptance [40]. All intermediate solutions are feasible, so the best plan found so far can be published at any time. The temperature decreases per iteration and all variants share the random seed, so every stopping rule follows the same search trajectory and differs only in the moment at which it stops.

4.2. Calibrated finish-time error

The error of task j is modelled as $e_j \sim \mathcal{N}(m(h_j), s(h_j)^2)$, where $h_j = f_j - t_0$ is its lead time, the nominal finish time minus the time t_0 of the planning snapshot, and

$$m(h) = b_0 + b_1 h, \qquad s(h) = s_0 + s_1 h, \tag{2}$$

because errors accumulate along a route. The parameters are fitted per disturbance level on development missions, the mean by least squares and the spread by least squares on the absolute residuals scaled by $\sqrt{\pi/2}$, which is less sensitive to the heavy tail of the errors. Under the mid and high disturbances of the main studies, both the mean and the spread grow with the lead time (Supplementary Table S2). The nominal model is therefore optimistic, and increasingly so along a route.

4.3. Chance-constrained planning model

The proposed model plans for the calibrated execution. A planned finish time f_j is accepted only if

$$f_j + m(f_j - t_0) + z\, s(f_j - t_0) \ \le\ d_j, \tag{3}$$

where $z \ge 0$ is a safety factor. The margin $m + zs$ depends on the finish time itself, so it is small for tasks served soon and large for tasks served late in a route. Because m and s are linear, the constraint reduces to an effective deadline,

$$f_j \le \tilde{d}_j, \qquad \tilde{d}_j = t_0 + \frac{d_j - t_0 - b_0 - z s_0}{1 + b_1 + z s_1}. \tag{4}$$

The repair and the search use $\tilde{d}_j$ instead of d_j, while the executor and the reward still use the true deadline.

Proposition 1. *Let $1 + b_1 + z s_1 > 0$. (i) A finish time satisfies Eq.* (3) *if and only if $f_j \le \tilde{d}_j$. (ii) Under the error model of Eq.* (2)*, a task planned to finish at $f_j \le \tilde{d}_j$ meets its deadline with probability at least $\Phi(z)$ if its route starts as planned. (iii) With $b_0 = b_1 = s_0 = s_1 = 0$, $\tilde{d}_j = d_j$ and the planning model is the nominal one.*

Proof. Substituting Eq. (2) into Eq. (3) gives $f_j(1 + b_1 + z s_1) \le d_j - b_0 - z s_0 + (b_1 + z s_1) t_0$, and dividing by the positive factor gives (i). For (ii), the actual finish is $F_j = f_j + e_j$ with $e_j \sim \mathcal{N}(m(h_j), s(h_j)^2)$ and $h_j = f_j - t_0$. By (i), $d_j - f_j - m(h_j) \ge z\, s(h_j)$, hence $\Pr[F_j \le d_j] = \Phi\big((d_j - f_j - m(h_j))/s(h_j)\big) \ge \Phi(z)$. Part (iii) is immediate.

The effective deadline turns a chance constraint on every task into a deterministic constraint of the same form as the original one. The insertion heuristics and the feasibility checks of the search are therefore reused unchanged, and computing $\tilde{d}_j$ costs one division per task. With $z = 0$, the model corrects the mean error only and plans for the median finish time. Larger values of z protect more tasks against lateness but leave fewer tasks feasible, so z trades the risk of a late service against the reward of attempting a task. It is chosen per disturbance level on development missions.

4.4. Nominal fallback for given-up tasks

A hard chance constraint has a side effect when the time windows are short compared with the spread of the error. If no vehicle can finish task j before $\tilde{d}_j$, even on a direct flight, the task is infeasible for the planner and is never attempted. A vehicle may still serve it in time with a probability below $\Phi(z)$ but well above zero, and it may have nothing better to do. The fallback keeps such tasks. Let $\tilde{H}$ be the set of tasks that no vehicle can finish by $\tilde{d}_j$ on a direct flight, and H the set of tasks that no vehicle can finish by d_j. The planning deadline is

$$d'_j = \begin{cases} d_j, & j \in \tilde{H} \setminus H, \\ \tilde{d}_j, & \text{otherwise.} \end{cases} \tag{5}$$

The constraints of all other tasks are unchanged, so every other task that is served keeps the guarantee of Proposition 1, while a fallback task carries none. Finding $\tilde{H}$ and H requires one direct-flight check per task and vehicle, which is negligible compared with the search. The fallback was designed after the held-out results of study 2 and is evaluated on new seeds in study 3.

4.5. Stopping rule and trigger

Every 5 ms, the stopping rule compares the predicted gain $\widehat{\Delta J}$ of searching $H = 10$ ms longer with the cost of doing so, and stops at time τ if

$$\widehat{\Delta J} \;\le\; C_{\mathrm{R}}(\tau) + \mu W H, \tag{6}$$

with a price of computation $\mu = 10^{-5}$ per millisecond. The gain is predicted from eleven features of the search history by a Poisson generalised linear model [41] fitted on development trajectories. The delay cost $C_{\mathrm{R}}(\tau) = \Lambda_{\mathrm{R}}(\tau + H) - \Lambda_{\mathrm{R}}(\tau)$ is the increase of the expected value of the tasks that a later start would lose,

$$\Lambda_{\mathrm{R}}(\tau) = \sum_j v_j\, \Phi\!\left(\frac{\sigma_j(\tau) + \bar{L} + m(h_j) - \mathrm{tol}_j}{s(h_j)} \right), \tag{7}$$

where $\sigma_j(\tau)$ is the start shift of the route of task j caused by waiting for the plan, tol_j the margin of the task to its deadline plus the waiting times before it on its route [37], $\bar{L}$ the expected latency and v_j the value of the task in Eq. (1). On the effective deadlines, the mean error is already part of the tolerance, so AnytimeSwarm-CC uses $m \equiv 0$. Λ_{R} does not decrease with τ, and it reduces to the exact loss of a deterministic plan when the spread vanishes (Supplementary Section S1). A hard budget B bounds every call. The replanner replans on every event and at least every $T_{\max} = 60$ s of mission time, a rolling-horizon policy in the terminology of rescheduling [26, 27].

Table 2: Disturbance and latency levels of tier B. The levels span mild to severe conditions and are not claimed to represent a particular vehicle.

Execution	accel. (m/s^2)	speed c.v.	wind (m/s)	service c.v.	Latency	median (s)	$\sigma_{\log}$	loss / timeout
low	3.0	0.03	2	0.05	low	0.2	0.5	–
mid	2.0	0.07	5	0.15	high	2.0	0.8	5% / 5 s
high	1.5	0.12	8	0.25				

5. Evaluation

5.1. Scenarios and kinematic tier

Missions are created by a seeded generator (Supplementary Table S1). The map is a square with a side of 10 km and the depot at its centre, half of the tasks are clustered around three hotspots, and half of them are known at the start while the others arrive in batches. Time windows have a slack of 180 to 600 s and services last 30 to 90 s. Three fleet scales with 5, 10 and 20 UAVs and 25, 50 and 140 tasks are combined with three event profiles, namely six arrival batches, three batches with two UAV failures, and three batches with four tightened deadlines.

Tier B executes each published plan in continuous mission time. Legs follow a trapezoidal speed profile with an acceleration limit, the airspeed of each leg deviates from the nominal speed by a multiplicative error, a constant wind of random direction changes the ground speed, and service durations are multiplied by a random factor (Table 2). The planner never sees these disturbances and receives the state of each UAV as telemetry. The dissemination latency is lognormal, and lost messages are resent after a timeout. All random draws use common random numbers [42] keyed to the mission seed, the UAV and the task, so two planners that assign the same task to the same UAV experience the same disturbance.

5.2. Software-in-the-loop tier

Tier C flies missions with three X500 quadrotors, each controlled by its own PX4 v1.17 autopilot in software-in-the-loop mode [13], in Gazebo Harmonic [14] on ROS 2 Jazzy [15] (Fig. 2). The mission manager runs the scenario timeline, builds the planning snapshot, calls the planner and releases the plan. The executor streams offboard setpoints to every autopilot at 10 Hz through a uXRCE-DDS bridge, flies each leg with a carrot that moves along the straight leg and holds the UAV at the task for the service duration. A tier C mission is a tier B scenario with three UAVs and fifteen tasks, rescaled by $S_X = 0.1$ in length and $S_T = 0.25$ in time. Speeds are multiplied by S_X/S_T, energy by S_X and the distance price by $1/S_X$, so every dimensionless ratio of the decision problem is preserved, and $\kappa_C = S_T \kappa_B$. The commanded ground speed of each leg follows the speed error and the scaled wind of tier B with the same random numbers, while acceleration, attitude dynamics, position tracking and communication are those of the simulated vehicles. Gazebo runs in lockstep with PX4 at twice real time, and mission time is simulation time.

5.3. Compared methods

All methods share the feasibility-first repair, the ALNS operators and parameters, the initial plan and the random seed, and every call has a hard budget of 250 ms. They differ in the stopping rule, the planning model and the trigger. Regret-2 repair without search is the constructive floor. ALNS is run with fixed budgets of 250 ms and 40 ms [4], the latter equal to the mean computation of AnytimeSwarm on development data, with a budget of 1 ms per open task, and with a stagnation rule that stops after 20% of the budget without improvement. Two stopping rules from the metareasoning literature are re-implemented with the same delay cost and price of computation as the proposed rule, so that only the prediction of the gain differs. The performance profile rule [10, 12] predicts the gain from a static profile over elapsed time and number of open tasks, and the online prediction rule [11] fits $q(t) = a - b\,e^{-ct}$ to the best objective found so far at

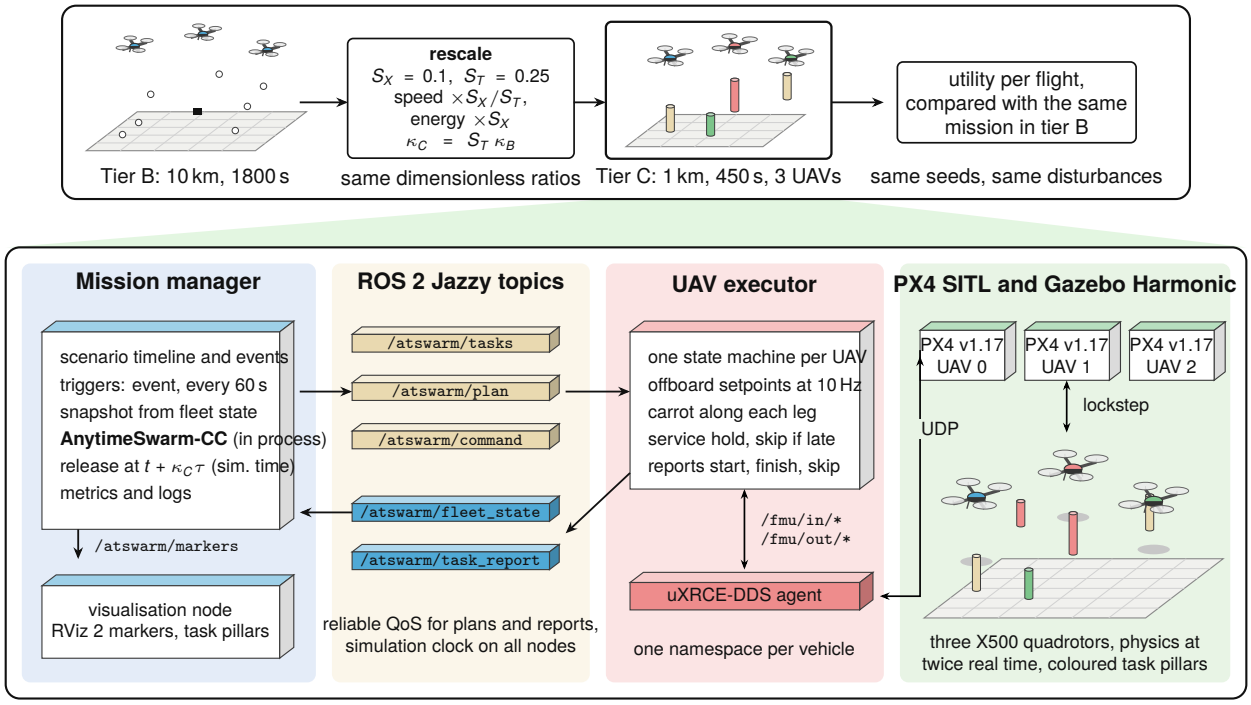


Figure 2: Software-in-the-loop tier. Top: a tier B mission is rescaled in length and time so that the decision problem is unchanged. Bottom: the planner runs inside the ROS 2 mission manager, the executor streams offboard setpoints through the uXRCE-DDS bridge to three PX4 autopilots, and Gazebo simulates three X500 quadrotors in lockstep.

every check. The constant-buffer model subtracts 60 s from every deadline, chosen on development missions among 30, 60 and 120 s, as is common for robust schedules with time buffers [6]. The proposed model uses Eq. (4) with $z = 0.5$ under mid and $z = 0$ under high disturbances, selected on development missions. All these methods use the periodic trigger. The conventional planner combines ALNS with 40 ms, the nominal model and the event trigger.

5.4. Pre-registered protocol

The evaluation consists of three studies, each with its own held-out seeds, written protocol and frozen code. Development used seeds 0–9 in tier B and 0–2 in tier C, on which the predictor, the compute price, z, the buffer and the error models were fixed. Before each study, the configurations, the models and the code were frozen with SHA-256 hashes, which the held-out runs checked before starting. Study 1 tested AnytimeSwarm-R against fixed and adaptive stopping rules on the nominal model (Supplementary Section S4).

Study 2 evaluates the chance-constrained model on held-out seeds 5000–5029 in tier B and 6000–6009 in tier C. Its matrix crosses two execution levels (mid, high), high latency, three latency factors ($\kappa_B = 10, 50, 200$) and two time-window factors (original and halved), which gives twelve configurations of 270 missions each, with the fifteen methods of Table 3. H1 states that the proposed replanner serves more reward than each of six baselines (ALNS with 40 ms, the size-scaled budget, online prediction, the performance profile, AnytimeSwarm-R and the constant buffer), and H2 compares it with the conventional planner. H3 tests whether the proposed stopping rule is equivalent to a fixed budget of 40 ms on the proposed model, with a margin of ±0.5 percentage points. In tier C, H4 compares the proposed replanner with ALNS with 40 ms and the conventional planner on 30 missions with mid disturbances and $\kappa_B = 50$. Study 3 was designed after the held-out results of study 2. Of two fallbacks compared on development seeds, the nominal one was selected by a rule stated beforehand. Study 3 uses new held-out seeds 7000–7029 in tier B and the matrix of study 2. Its primary family compares the fallback variant with the replanner of study 2 over all configurations (H5),

with the constant buffer under high disturbances with halved windows (H6), and with ALNS with 40 ms (H7) and the constant buffer (H8) over all configurations.

In all studies, the utility difference of two methods is averaged per mission over the configurations and tested with a paired bootstrap [43] of 10 000 resamples over missions. Primary families are corrected with the Holm procedure [44], and equivalence is tested with two one-sided tests. All other comparisons are descriptive. A tier C run in which a vehicle does not take off in time, or whose real-time factor leaves $[1.6, 2.2]$, is repeated once. Tier B runs four missions in parallel on an Intel Core i7-11800H laptop, with the methods of a configuration run in turn. A separately written validator found no infeasible plan among 4 779 development plans. The protocols and their deviations are available with the code.

6. Results

6.1. Study 1 in brief: execution-aware stopping on the nominal model

Study 1 made only the stopping rule execution-aware and kept the nominal planning model. In 34 560 tier B runs, AnytimeSwarm-R needed 15% less computation per call than AnytimeSwarm, but its advantage over ALNS with 40 ms was +0.32 [+0.01, +0.63] percentage points (pp) with a Holm-adjusted p of 0.128, and its advantages over AnytimeSwarm and the stagnation rule were not significant either. Periodic replanning raised the utility of every method by +1.29 to +1.56 pp. Supplementary Sections S3 and S4 report these results in full, together with an exact-execution benchmark, tier C flights and a scalability study with 40 UAVs. Study 2 was designed after these results and moves the error model from the decision to stop into the planning model.

6.2. Study 2: the chance-constrained model against baselines

Study 2 comprises 48 600 tier B runs, that is, 270 held-out missions in 12 configurations with 15 methods (Table 3). The proposed replanner reached the highest utility, 52.15%. Its advantage over the six baselines of H1 ranged from +1.35 pp over the constant buffer to +2.87 pp over ALNS with 40 ms, and every Holm-adjusted p-value was below 0.001, so H1 holds for all six comparisons. In relative terms, it served 2.6% more reward than the constant buffer and 5.5% more than online performance prediction. Against the conventional planner, the advantage was +4.44 [+3.82, +5.05] pp, or 9.3%, so H2 holds as well.

The advantage comes from fewer late tasks. The proposed replanner served 5.72 tasks per mission after their deadline, against 11.22 for ALNS with 40 ms, that is, 49% fewer, and 6.53 for the constant buffer. It lost no task to a late plan, whereas ALNS with 40 ms lost 3.58 and the full budget of 250 ms 11.58 tasks per mission. On the nominal model, the stopping rules hardly differed. ALNS with 40 ms, the size-scaled budget, the performance profile, online prediction, AnytimeSwarm and AnytimeSwarm-R lay between 49.28% and 49.66%, although their computation ranged from 11.2 to 58.1 ms per call (Supplementary Fig. S4).

The ablations confirm that the planning model carries the gain (Table 3, bottom). With a fixed budget of 40 ms instead of the proposed stopping rule, utility changed by +0.22 [+0.07, +0.37] pp. The 90% interval of this difference, +0.22 [+0.09, +0.35] pp, lies inside the margin of ±0.5 pp, so the stopping rule is equivalent to the fixed budget on the chance-constrained model (H3). It still guards against long budgets on slow planners. At $\kappa_B = 200$, the full budget lost 23.42 tasks per mission to late plans and served +11.34 [+10.78, +11.91] pp less reward than the proposed replanner. With the event trigger instead of the periodic one, the proposed replanner lost +0.22 [+0.06, +0.38] pp, far less than the +1.29 to +1.56 pp that periodic replanning added on the nominal model of study 1. With $z = 0$, the difference was +0.18 [−0.06, +0.41] pp and not significant.

The gain was not uniform. Per configuration, the proposed replanner was better than the constant buffer in 9 of the 12 configurations and worse in 3. All losses occurred under high disturbances with halved windows, where it served −2.01 [−2.80, −1.21] pp relative to the buffer and −1.57 [−2.52, −0.64] pp relative to ALNS with 40 ms. With short windows and large errors, the effective deadline of many tasks falls before the earliest

Table 3: Tier B, held-out missions of study 2: 270 missions × 12 configurations per method. Utility, tasks served late (Late), tasks lost because a plan arrived too late (Lost), planning calls and computation per call; difference of the proposed method minus each method, pooled over configurations (percentage points, 95% paired bootstrap interval; Holm-adjusted p for the six primary comparisons, marked *).

Method	Utility (%)	Late	Lost	Calls	ms/call	Proposed minus method (pp)
Proposed	52.15	5.72	0.00	23.3	61.0	–
Conventional (40 ms, event-only)	47.71	10.31	3.32	6.8	40.2	+4.44 [+3.82, +5.05]
Regret-2 repair (no search)	47.95	11.19	2.34	23.2	0.5	+4.21 [+3.57, +4.84]
ALNS, fixed 250 ms [4]	46.23	9.78	11.58	22.9	250.1	+5.92 [+5.41, +6.44]
ALNS, fixed 40 ms [4]	49.28	11.22	3.58	23.3	40.1	+2.87 [+2.29, +3.45], p <0.001*
ALNS, size-scaled budget	49.58	11.18	3.75	23.3	17.7	+2.57 [+1.99, +3.17], p <0.001*
ALNS, stagnation stop	49.04	10.98	5.50	23.2	55.8	+3.11 [+2.54, +3.69]
Performance profile [10, 12]	49.54	11.24	3.52	23.2	30.2	+2.62 [+2.04, +3.19], p <0.001*
Online performance prediction [11]	49.45	11.17	2.75	23.2	11.2	+2.70 [+2.13, +3.28], p <0.001*
AnytimeSwarm	49.31	11.17	3.89	23.2	58.1	+2.84 [+2.29, +3.40]
AnytimeSwarm-R	49.66	11.21	3.73	23.2	50.9	+2.49 [+1.92, +3.06], p <0.001*
Constant 60 s buffer [6]	50.81	6.53	0.00	23.4	40.1	+1.35 [+0.93, +1.77], p <0.001*
Ablations of the proposed method						
Proposed, event-only trigger	51.93	5.85	0.00	6.8	57.8	+0.22 [+0.06, +0.38]
Proposed model, fixed 40 ms stop	51.93	5.74	0.00	23.4	40.1	+0.22 [+0.07, +0.37]
Proposed, mean correction only	51.97	6.84	0.00	23.3	58.8	+0.18 [−0.06, +0.41]

possible arrival, so the planner gives up tasks that a less cautious plan still serves in time. This weakness motivated the fallback of Section 4.4.

6.3. Study 3: nominal fallback for given-up tasks

Study 3 comprises 16 200 tier B runs on 270 new held-out missions in 12 configurations with five methods (Table 4, Fig. 3). It first replicates study 2. The proposed replanner of study 2 served +1.62 [+1.22, +2.01] pp more reward than the constant buffer over all configurations, and it again lost to the buffer under high disturbances with halved windows, by −1.95 [−2.72, −1.20] pp. The weakness is therefore systematic and not a property of the seeds of study 2.

The fallback variant reached a utility of 54.40%, the highest of all methods, and all four primary comparisons were significant (Holm-adjusted $p < 0.001$). It served +1.97 [+1.61, +2.33] pp more reward than the proposed replanner of study 2 (H5), that is, 3.8% more. Under high disturbances with halved windows, it served +3.31 [+2.58, +4.03] pp more than the constant buffer (H6). Over all configurations, it served +4.61 [+4.15, +5.06] pp more than ALNS with 40 ms (H7, 9.3%) and +3.59 [+3.17, +4.02] pp more than the constant buffer (H8, 7.1%). Against the conventional planner, the advantage was +6.19 [+5.69, +6.69] pp, or 12.8%. The gain over the replanner of study 2 was concentrated where the chance constraint gives up tasks, with +5.25 [+4.49, +6.05] pp under high disturbances with halved windows. With the original windows, the two variants were close (+0.52 [−0.01, +1.06] pp under mid and +0.11 [−0.68, +0.89] pp under high disturbances).

The fallback has a price in punctuality. A fallback task is planned against its nominal deadline and is often served late. The fallback variant served 8.33 tasks late per mission, 46% more than the 5.70 of the replanner of study 2, although still 25% fewer than ALNS with 40 ms. It also lost 1.55 tasks per mission to late plans, against none for the replanner of study 2. A late service yields no reward, so these attempts cost only flight time. Where a late arrival has a cost of its own, the guarantee of Proposition 1 without fallback remains the safer choice.

Table 4: Study 3, tier B held-out results: utility (%) per method overall and per regime (disturbance level and time-window factor, mean over κ_B), tasks served late per mission and computation per call. 270 missions, 12 configurations.

		Mid		High			
Method	All	×1	×0.5	×1	×0.5	Late	ms/call
Proposed with fallback	54.40	72.08	49.89	56.39	39.22	8.33	55.0
Proposed (study 2)	52.42	71.56	47.88	56.28	33.97	5.70	57.4
Constant 60 s buffer	50.80	69.33	45.48	52.48	35.92	6.62	40.1
ALNS, fixed 40 ms	49.79	67.38	46.05	50.10	35.61	11.18	40.1
Conventional (40 ms, event-only)	48.20	65.83	44.46	48.61	33.91	10.27	40.1

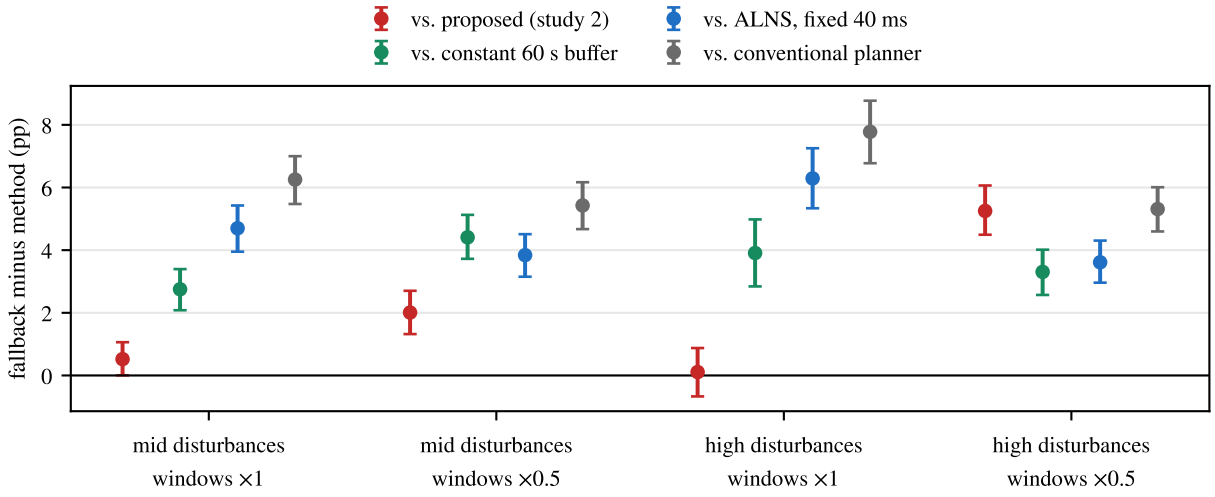


Figure 3: Study 3, tier B, held-out seeds 7000–7029: utility of the replanner with nominal fallback minus each other method, per regime (mean over κ_B, 270 missions, 95% paired bootstrap intervals).

6.4. *Software-in-the-loop flights*

All 120 held-out flights of study 2 were completed (Table 5). As specified in the protocol, 46 runs with a real-time factor outside the validity range were repeated once. Because the laptop was also used for other work, many repeats ran slow as well, and two sensitivity analyses that were not pre-registered, with the original runs and with valid pairs only, led to the same conclusions (Supplementary Section S5).

The proposed replanner reached 59.89%, against 60.15% for ALNS with 40 ms and 58.61% for the conventional planner. The paired differences were −0.25 [−4.02, +3.57] pp and +1.28 [−2.20, +4.64] pp, with Holm-adjusted p-values of 0.942 and 0.942, so H4 is not supported. The mechanism of tier B reappeared in the counts. The proposed replanner served 0.90 tasks late per mission, against 1.20 for ALNS with 40 ms, which is 25% fewer, and it skipped no task at departure, whereas ALNS with 40 ms skipped 0.23 and the conventional planner 0.57 tasks per mission.

The same missions were also run in tier B with the rescaled parameters. Tier B predicted the utility of a flight closely ($r = 0.86$ over 120 runs), but even tier B did not resolve the differences between the methods on these 30 missions (+2.82 [−1.20, +6.62] pp to ALNS with 40 ms). With a per-mission standard deviation of 10.9 pp in tier C, confirming a difference of 2.8 pp with a power of 80% would require about 118 missions per method. Tier C thus shows that the replanner runs on autopilots with the same computation and fewer late tasks, but it cannot confirm the size of the gain measured in tier B.

Table 5: Tier C (PX4 SITL, Gazebo, ROS 2), study 2: 30 held-out missions per method, mid disturbances, $\kappa_B = 50$. Utility, tasks served late, tasks skipped at departure, planning calls and computation per call; tier B utility on the same rescaled missions.

Method	Utility C (%)	Late	Skipped	Calls	ms/call	Utility B (%)
Proposed	59.89	0.90	0.00	21.9	44.4	61.50
ALNS, fixed 40 ms	60.15	1.20	0.23	20.9	40.2	58.67
Constant 60 s buffer	57.26	0.83	0.00	21.0	40.2	59.60
Conventional (40 ms, event-only)	58.61	1.37	0.57	6.3	40.2	59.70

7. Discussion and limitations

The three studies suggest an order of priorities for a multi-UAV replanner under uncertain execution. The planning model matters most. Moving every deadline by the predicted finish-time error raised the utility by more than any choice of stopping rule or trigger, and it removed the losses from late plans. The trigger comes second. Periodic replanning helped every method on the nominal model, but only by 0.22 pp on the chance-constrained model, which suggests that both correct the same accumulated deviations, one before and one after they occur. The stopping rule comes last. On the proposed model, it was equivalent to a matched fixed budget, and its value lies in avoiding the losses of long budgets on slow planners. The metareasoning rules behaved in the same way, probably because the improvement curve of this ALNS flattens within a few tens of milliseconds. Monitoring rules may matter more for searches whose improvement continues for longer.

The proposed model and the constant buffer both shift deadlines forward, and the buffer was the strongest baseline. The difference lies in the shape of the margin. A constant margin is too large for tasks reached soon and too small for tasks at the end of a route, whereas the proposed margin grows with the lead time. A hard chance constraint, however, discards every task whose probability of being served on time falls below $\Phi(z)$, even when a vehicle has nothing better to do. Study 3 confirmed this explanation on new seeds. Keeping the given-up tasks with their nominal deadline removed the weakness and left the guarantee of the other tasks intact, but many fallback tasks were served late. A soft formulation that weights every task by its probability of being served on time would make this trade explicit and could be tuned to the cost of a late arrival in a given application.

The two tiers play different roles. Tier B provides the number of missions that a paired comparison needs, and tier C checks that its conclusions survive an autopilot, a physics engine and a real middleware. In both studies with flights, tier B predicted the utility of individual flights closely, and tier C reproduced the mechanism but could not resolve differences of a few percentage points. A two-tier study should therefore plan its number of flights from the smallest difference that it is meant to confirm.

Several limitations bound these conclusions. The error model is linear in the lead time and fitted per disturbance level, so the level is assumed to be known, and z was selected per level on development missions. The chance constraint is imposed on every task separately rather than jointly on a route. The fallback was designed after the held-out results of study 2 and was tested on new seeds, but only in tier B. Tier C uses three UAVs and fifteen tasks per mission, and many of its runs ran slower than intended because the laptop was shared with other work. Software-in-the-loop simulation reproduces the autopilot and the vehicle dynamics but not the sensors, the radio link or the weather of a field test, and the wind is constant during a mission. The latency factor emulates a slower planning computer instead of measuring one. Because the stopping rules read a wall clock, repeated runs are not bit-identical, and the mean utility of a method varied by up to 0.12 pp between two runs of the same missions. Finally, all methods share one ALNS and one family of scenarios, and other search methods or mission types may change the size of the gains.

8. Conclusion

This paper proposed AnytimeSwarm-CC, a multi-UAV replanner that plans against a chance constraint on the finish time of every task. Because the calibrated finish-time error grows linearly with the lead time, the chance constraint reduces exactly to an effective deadline per task, and fast heuristics are reused unchanged. On held-out missions of pre-registered studies, the chance-constrained model served 1.35 to 2.87 percentage points more reward than six baselines, including metareasoning stopping rules and a constant buffer, and a nominal fallback for the tasks that the constraint gives up added 1.97 percentage points on new seeds, at the price of more late tasks. The gain came from the planning model, while the stopping rule was equivalent to a well-chosen fixed budget. Software-in-the-loop flights with PX4 and Gazebo reproduced the reduction of late tasks but were too few to confirm the utility gain. Future work will replace the fallback by an objective that weights every task by its probability of being served on time, fly the fallback variant on autopilots with enough missions for the expected effect, adapt the safety factor online, and run the replanner on embedded hardware.

CRediT authorship contribution statement

Cao-Phuc Ha: Conceptualization, Methodology, Software, Investigation, Formal analysis, Writing – original draft. Anh Nguyen Vu Minh: Software, Validation, Writing – review and editing. Phu-Nguyen Le: Supervision, Methodology, Writing – review and editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data and code availability

The planner, both simulation tiers, the experiment configurations with their SHA-256 hashes, the pre-registered protocol, all raw results and the scripts that produce every number, table and figure of this paper will be made available in a public repository upon publication.

Acknowledgements

The authors would like to express their sincere appreciation to the Vietnam Aerospace University for supporting this research during the 2026–2027 academic year.

References

[1] B. P. Gerkey, M. J. Matarić, A formal analysis and taxonomy of task allocation in multi-robot systems, The International Journal of Robotics Research 23 (9) (2004) 939–954. doi:10.1177/0278364904045564.

[2] V. Pillac, M. Gendreau, C. Guéret, A. L. Medaglia, A review of dynamic vehicle routing problems, European Journal of Operational Research 225 (1) (2013) 1–11. doi:10.1016/j.ejor.2012.08.015.

[3] H. N. Psaraftis, M. Wen, C. A. Kontovas, Dynamic vehicle routing problems: Three decades and counting, Networks 67 (1) (2016) 3–31. doi:10.1002/net.21628.

[4] S. Ropke, D. Pisinger, An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows, Transportation Science 40 (4) (2006) 455–472. doi:10.1287/trsc.1050.0135.

[5] F. Errico, G. Desaulniers, M. Gendreau, W. Rei, L.-M. Rousseau, A priori optimization with recourse for the vehicle routing problem with hard time windows and stochastic service times, European Journal of Operational Research 249 (1) (2016) 55–66. doi:10.1016/j.ejor.2015.07.027.

[6] A. Agra, M. Christiansen, R. Figueiredo, L. M. Hvattum, M. Poss, C. Requejo, The robust vehicle routing problem with time windows, Computers & Operations Research 40 (3) (2013) 856–866. doi:10.1016/j.cor.2012.10.002.

[7] S. S. Ponda, L. B. Johnson, J. P. How, Distributed chance-constrained task allocation for autonomous multi-agent teams, in: Proceedings of the American Control Conference (ACC), 2012, pp. 4528–4533. doi:10.1109/ACC.2012.6315626.

[8] T. Dean, M. Boddy, An analysis of time-dependent planning, in: Proceedings of the Seventh National Conference on Artificial Intelligence (AAAI-88), AAAI Press, 1988, pp. 49–54.

[9] S. Russell, E. Wefald, Do the Right Thing: Studies in Limited Rationality, MIT Press, Cambridge, MA, 1991. doi:10.7551/mitpress/2474.001.0001.

[10] S. Zilberstein, Using anytime algorithms in intelligent systems, AI Magazine 17 (3) (1996) 73–83. doi:10.1609/aimag.v17i3.1232.

[11] J. Svegliato, K. H. Wray, S. Zilberstein, Meta-level control of anytime algorithms with online performance prediction, in: Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence (IJCAI-18), IJCAI Organization, 2018, pp. 1499–1505. doi:10.24963/ijcai.2018/208.

[12] M. Boddy, T. L. Dean, Deliberation scheduling for problem solving in time-constrained environments, Artificial Intelligence 67 (2) (1994) 245–285. doi:10.1016/0004-3702(94)90054-X.

[13] L. Meier, D. Honegger, M. Pollefeys, PX4: A node-based multithreaded open source robotics framework for deeply embedded platforms, in: Proceedings of the IEEE International Conference on Robotics and Automation (ICRA), 2015, pp. 6235–6240. doi:10.1109/ICRA.2015.7140074.

[14] N. Koenig, A. Howard, Design and use paradigms for Gazebo, an open-source multi-robot simulator, in: Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2004, pp. 2149–2154. doi:10.1109/IROS.2004.1389727.

[15] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, W. Woodall, Robot operating system 2: Design, architecture, and uses in the wild, Science Robotics 7 (66) (2022) eabm6074. doi:10.1126/scirobotics.abm6074.

[16] M. M. Solomon, Algorithms for the vehicle routing and scheduling problems with time window constraints, Operations Research 35 (2) (1987) 254–265. doi:10.1287/opre.35.2.254.

[17] P. Vansteenwegen, W. Souffriau, D. Van Oudheusden, The orienteering problem: A survey, European Journal of Operational Research 209 (1) (2011) 1–10. doi:10.1016/j.ejor.2010.03.045.

[18] S. Wang, W. Sun, M. Huang, An adaptive large neighborhood search for the multi-depot dynamic vehicle routing problem with time windows, Computers & Industrial Engineering 191 (2024) 110122. doi:10.1016/j.cie.2024.110122.

[19] H.-L. Choi, L. Brunet, J. P. How, Consensus-based decentralized auctions for robust task allocation, IEEE Transactions on Robotics 25 (4) (2009) 912–926. doi:10.1109/TRO.2009.2022423.

[20] S. Choudhury, J. K. Gupta, M. J. Kochenderfer, D. Sadigh, J. Bohg, Dynamic multi-robot task allocation under uncertainty and temporal constraints, Autonomous Robots 46 (1) (2022) 231–247. doi:10.1007/s10514-021-10022-9.

[21] H. Huang, Z. Jiang, T. Yan, Y. Bai, Dynamic task allocation for heterogeneous multi-UAVs in uncertain environments based on 4DI-GWO algorithm, Drones 8 (6) (2024) 236. doi:10.3390/drones8060236.

[22] C. Qian, B. Fu, Uncertainty-gated distributed dynamic task allocation for multi-UAV systems under intermittent communication, Drones 10 (10) (2026) 782. doi:10.3390/drones10100782.

[23] S.-Y. Hwang, S. Oh, J.-C. Lee, S. Lee, J.-H. Lee, H.-J. Hwang, Assignment-preserving replanning for dynamic task allocation in multi-UAV systems, Applied Sciences 16 (19) (2026) 9414. doi:10.3390/app16199414.

[24] Y. Shen, T. Hu, Y. Wan, H. Sun, M. Tan, Edge-responsive local-repair particle swarm optimization for dynamic spatiotemporal task allocation of IoT-connected UAV swarms, IEEE Access 14 (2026) 115169–115190. doi:10.1109/ACCESS.2026.3717031.

[25] Y. Cong, Y. Li, X. Zhu, Z. Wang, An event-triggered path-based bid assignment method for energy-aware dynamic multi-UAV task allocation, Drones 10 (9) (2026) 684. doi:10.3390/drones10090684.

[26] G. E. Vieira, J. W. Herrmann, E. Lin, Rescheduling manufacturing systems: A framework of strategies, policies, and methods, Journal of Scheduling 6 (1) (2003) 39–62. doi:10.1023/A:1022235519958.

[27] D. Ouelhadj, S. Petrovic, A survey of dynamic scheduling in manufacturing systems, Journal of Scheduling 12 (4) (2009) 417–431. doi:10.1007/s10951-008-0090-8.

[28] Á. Calvo, J. Capitán, Heterogeneous multirobot task allocation for long-endurance missions in dynamic scenarios, IEEE Transactions on Robotics 41 (2025) 6494–6513. doi:10.1109/TRO.2025.3626651.

[29] M. A. Ali, U. Athar, A. Maqsood, SITL validation of federated reinforcement learning for priority-based dynamic task allocation in UAV swarms, Expert Systems 43 (12) (2026) e70459. doi:10.1111/exsy.70459.

[30] M. Gendreau, G. Laporte, R. Séguin, Stochastic vehicle routing, European Journal of Operational Research 88 (1) (1996) 3–12. doi:10.1016/0377-2217(95)00050-X.

[31] R. W. Bent, P. Van Hentenryck, Scenario-based planning for partially dynamic vehicle routing with stochastic customers, Operations Research 52 (6) (2004) 977–987. doi:10.1287/opre.1040.0124.

[32] E. A. Hansen, S. Zilberstein, Monitoring and control of anytime algorithms: A dynamic programming approach, Artificial Intelligence 126 (1–2) (2001) 139–157. doi:10.1016/S0004-3702(00)00068-0.

[33] Y. Sung, L. P. Kaelbling, T. Lozano-Pérez, Learning when to quit: Meta-reasoning for motion planning, in: Proceedings of the 2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), IEEE, 2021, pp. 4692–4699. doi:10.1109/IROS51168.2021.9636864.

[34] M. Cashmore, A. Coles, B. Cserna, E. Karpas, D. Magazzeni, W. Ruml, Temporal planning while the clock ticks, in: Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS), Vol. 28, AAAI Press, 2018, pp. 39–46. doi:10.1609/icaps.v28i1.13878.

[35] S. S. Shperberg, A. Coles, E. Karpas, W. Ruml, S. E. Shimony, Situated temporal planning using deadline-aware metareasoning, in: Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS), Vol. 31, AAAI Press, 2021, pp. 340–348. doi:10.1609/icaps.v31i1.15979.

[36] A. Dionne, J. Thayer, W. Ruml, Deadline-aware search using on-line measures of behavior, in: Proceedings of the International Symposium on Combinatorial Search (SoCS), Vol. 2, AAAI Press, 2011, pp. 39–46. doi:10.1609/socs.v2i1.18199.

[37] M. W. P. Savelsbergh, The vehicle routing problem with time windows: Minimizing route duration, ORSA Journal on Computing 4 (2) (1992) 146–154. doi:10.1287/ijoc.4.2.146.

[38] J.-Y. Potvin, J.-M. Rousseau, A parallel route building algorithm for the vehicle routing and scheduling problem with time windows, European Journal of Operational Research 66 (3) (1993) 331–340. doi:10.1016/0377-2217(93)90221-8.

[39] P. Shaw, Using constraint programming and local search methods to solve vehicle routing problems, in: Principles and Practice of Constraint Programming – CP98, Vol. 1520 of Lecture Notes in Computer Science, Springer, 1998, pp. 417–431. doi:10.1007/3-540-49481-2_30.

[40] S. Kirkpatrick, C. D. Gelatt, M. P. Vecchi, Optimization by simulated annealing, Science 220 (4598) (1983) 671–680. doi:10.1126/science.220.4598.671.

[41] P. McCullagh, J. A. Nelder, Generalized Linear Models, 2nd Edition, Chapman and Hall, London, 1989. doi:10.1007/978-1-4899-3242-6.

[42] A. M. Law, Simulation Modeling and Analysis, 5th Edition, McGraw-Hill Education, New York, 2015.

[43] B. Efron, R. J. Tibshirani, An Introduction to the Bootstrap, Chapman and Hall, New York, 1993. doi:10.1007/978-1-4899-4541-9.

[44] S. Holm, A simple sequentially rejective multiple test procedure, Scandinavian Journal of Statistics 6 (2) (1979) 65–70.